

The Java Virtual Machine

Norman Matloff
University of California at Davis
© 2001, 2002, N. Matloff

August 28, 2002

Contents

1	Overview	2
2	The JVM Architecture	2
2.1	Registers	3
2.2	Memory Areas	3
2.3	The Instruction Set	4
2.3.1	Structure	4
2.3.2	Example	4
2.3.3	Some Further Examples of JVM Instructions	8
3	References	8

1 Overview

You may have heard about the **Java virtual machine** (JVM), associated with the Java language. What is really going on here?

The name of any Java source file has a `.java` suffix, just like C source file names end in `.c`. Suppose for example our Java program source code is all in one file, `x.java`, and that for instance we are doing our work on a PC running Linux.

We first compile, using the Java compiler, **javac**:

```
javac -g x.java
```

(The `-g` option saves the symbol table for use by a debugger.)

This produces the executable Java file, `x.class`, which contains machine language, called **byte code**, to run on a “Java machine.” But we don’t have such a machine.

Actually, JVM chips — i.e. chips that run Java byte code — do exist, but they are not in common use. Instead, we have a program that emulates the operation of such a machine. This, of course, is the reason for the ‘V’ in “JVM.” The emulator (**interpreter**) program is named **java**. Note that in our case here, **java** will be a program running the Intel machine language of our PC.¹

We then run our program:

```
java x
```

Note that Java not only runs on a virtual machine, it also is in some sense running under a virtual operating system. For example, the analog of C’s **printf()** function in Java is **System.out.println()**. Recall that (if our real machine is running UNIX) a **printf()** call in a C program calls the **write()** function in the OS. But this does not happen in the Java case; our OS is running on our real machine, but our Java program is running on the JVM. What actually happens is that `System.out.println()` makes a call to what amounts to an OS in the JVM, and the latter calls **write()** on the real machine.

In what follows, it is assumed that the reader has at least a rudimentary knowledge of Java. See the author’s Java tutorial at <http://heather.cs.ucdavis.edu/~matloff/java.html> for a 30-minute introduction to the Java language.

2 The JVM Architecture

The JVM is basically a stack-based machine, with a 32-bit word size, using 2s complement arithmetic.

¹would be an Intel machine-language program too.

2.1 Registers

The JVM register set² is fairly small:

- **pc**: program counter
- **optop**: pointer to the top of the operand stack (a kind of work space; see below) for the currently-active method
- **frame**: pointer to the stack frame of the currently-active method
- **vars**: pointer to the beginning of the local variables of the currently-active method

2.2 Memory Areas

- the Stack:

The JVM architecture is very much stack-oriented. Most instructions access the stack in some way. The stack is also used for method calls, as with many classical machine architectures.

A method call produces a new stack frame, which is pushed onto the Stack for that program.³ and a return pops the frame from the program's Stack.

A stack frame is subdivided into the following.

- a Local Variables section:

All the local variables and arguments for the method are stored here, one per **slot** (i.e. one variable per word), with the arguments stored first and then the locals. The arguments and locals are stored in order of their declaration. In the case in which the method is an instance method, i.e. is not declared **static**, slot 0 will contain a pointer to the object on which this method is operating, i.e. the object addressed as **this** in the program's source code.

- an Operand Stack section:

This is the area on which the method's instructions operate. Almost all JVM instructions are stack-based; e.g. an "add" instruction pops the top two elements of the stack, adds them, and pushes the sum back onto the stack. So the Operand Stack portion of a method's stack frame is what we are referring to when we refer to an instruction as operating on "the" stack.

- a Frame Data section:

We will not go into the details of this.

- the Method Area:

The classes used by the executing program are stored here. This includes:

- the bytecode and access types of the methods of the class (similar to the .text section of a UNIX program, except for the access types)

²Remember, in the case of a real Java chip, these would be real registers, but in the JVM setting, these registers, as well as the instruction set, are simulated by the **java** program.

³More precisely, for that thread, since many Java programs are threaded.

- the static variables of the class (their values and access types, i.e **public**, **private** etc.)

The **pc** register points to the location within the Method Area of the JVM instruction to be executed next.

The Method Area also includes the Constant Pool. It contains the string and numeric literals used by the program, e.g. the 1.2 in

```
float W = 1.2;
```

and also contains information on where each method is stored in the Method Area.

- the Heap:

This is where Java objects exist. Whenever the Java **new** operation is invoked to create an object, the necessary memory is allocated within the heap.⁴ This space will hold the instance variables for the object, and a pointer to the location of the object's class in the Method Area.

2.3 The Instruction Set

2.3.1 Structure

Most instructions are a single byte in length, consisting only of an op code, but there are a few multi-byte instructions as well. As mentioned earlier, almost all JVM instructions involve operand stack operations.

2.3.2 Example

Consider the following source code, Minimum.java:

```
public class Minimum {  
  
    public static void main(String[] Args)  
  
    {   int X,Y,Z;  
  
        X = Integer.parseInt(Args[0]);  
        Y = Integer.parseInt(Args[1]);  
        Z = Min(X,Y);  
        System.out.println(Z);  
    }  
}
```

⁴Just as in C, where a call to `malloc()` results in memory space being allocated from the heap, and just as in C++, where a call to **new** results in space being taken from the heap.

```

public static int Min(int U, int V)

{
    int T;

    if (U < V) T = U;
    else T = V;
    return T;
}

```

We use the compiler, **javac**, to produce the class file, `Minimum.class`. The latter is what is executed, when we run the Java interpreter, **java**.

We can use another program, **javap**, to disassemble the contents of `Minimum.class`:⁵

```

% javap -c Minimum
Compiled from Minimum.java
public class Minimum extends java.lang.Object {
    public Minimum();
    public static void main(java.lang.String[]);
    public static int Min(int, int);
}

Method Minimum()
  0 aload_0
  1 invokespecial #1 <Method java.lang.Object()>
  4 return

Method void main(java.lang.String[])
  0 aload_0
  1 iconst_0
  2 aaload
  3 invokestatic #2 <Method int parseInt(java.lang.String)>
  6 istore_1
  7 aload_0
  8 iconst_1
  9 aaload
 10 invokestatic #2 <Method int parseInt(java.lang.String)>
 13 istore_2
 14 iload_1
 15 iload_2
 16 invokestatic #3 <Method int Min(int, int)>

```

⁵So, this listing here is similar to the output of **gcc -S** in a C/C++ context.

```

19 istore_3
20 getstatic #4 <Field java.io.PrintStream out>
23 iload_3
24 invokevirtual #5 <Method void println(int)>
27 return

```

Method int Min(int, int)

```

0 iload_0
1 iload_1
2 if_icmpge 10
5 iload_0
6 istore_2
7 goto 12
10 iload_1
11 istore_2
12 iload_2
13 ireturn

```

Note that each “line number” is actually an offset, i.e. the distance in bytes of the given instruction from the beginning of the given method.

Let’s first look at the Local Variables section of Main()’s stack frame:

slot	variable
0	pointer to Args
1	X
2	Y
3	Z

Now consider the call to Min. The code

```
Z = Min(X,Y);
```

gets compiled to

```

14 iload_1
15 iload_2
16 invokestatic #3 <Method int Min(int, int)>
19 istore_3

```

As in a classical architecture, the arguments for a call will be pushed onto Main()’s Operand Stack, as follows. The **iload_1** (“integer load”) instruction pushes slot 1 to the operand stack. Since slot 1 in main()

contains X,⁶ this means that X will be pushed onto the operand stack. The instruction in offset 15 will then push Y.

If `Min()` had been an instance function, i.e. not declared **static**, the first argument pushed would have been **this**, a pointer to the object on which `Min()` is being invoked.

The call itself is then done by the instruction **invokestatic** in offset 16. (For a nonstatic method call, we would use **invokevirtual**.) This instruction is three bytes in length, as can be seen by the fact that the following instruction begins at offset 19. The instruction's two-byte operand, in this case 3, serves as a pointer to an entry corresponding to `Min()` in the Constant Pool of the Method Area. In this way, the JVM will know where the first instruction of `Min()` is located, and the **pc** will be set accordingly, causing `Min()` to begin execution.

The actions of **invokestatic** is to pop the arguments off the caller's (in this case, `Main()`'s) Operand Stack,⁷ and place them in the Local Variables section of the callee's (in this case `Min()`'s) Stack.

The **istore_3** instruction following the call, in offset 19, pops the top of `Main()`'s Operand Stack and places it into slot 3, in our case Z.⁸

The bytecode in `Min()` is similar. The main new instruction here is **if_icmpge** ("if integer compare greater-than-or-equal") in offset 2. Let's refer to the top element of the current (i.e. `Min()`'s) Operand Stack as *op2* and the next-to-top element as *op1*. The instruction pops these two elements off the Operand Stack, compares them, and then jumps to the branch target if $op1 \geq op2$. Again, keep in mind that these items on `Min()`'s Operand Stack were placed there by the **iload_1** and **iload_2** instructions in `Min()`, which took them from `Min()`'s Local Variables area, and they in turn had been placed there by `Main()` when executing **invokestatic**.

The branch target is specified as the distance from the current instruction to the target. As can be seen in the JVM assembler code above, our target is offset 10 (an **iload_1** instruction). Since our **if_icmpge** instruction is in offset 2, the distance will be 8, i.e. 0x0008. Those latter two bytes comprise the second and third bytes of the instruction.

`Min()`'s **ireturn** instruction then pops the current (i.e. `Min()`'s) Operand Stack and pushes the popped value on top of the caller's (i.e. `Main()`'s) Operand Stack.

From the Sun JVM specifications (see below), we know that the op code for the **if_icmpge** instruction is 0xa2. Thus the entire instruction should be 0xa20008, and this string of three bytes should appear in the file `Minimum.class`. Running the command

```
od -t x1 Minimum.class
```

on a UNIX machine, we see that 0xa20008 is indeed in that file, in bytes 1255-1257 octal (685-687 decimal).

⁶Slot 0 contains the argument to `main()`, `Args`.

⁷Note that this means that the JVM needs to know how many arguments the callee has, information it can obtain from the Method Area. Also note that in this was the JVM does its own stack cleanup, unlike what we've seen in other machines.

⁸Explained below, the value now popped had been placed there by the **ireturn** instruction in `Min()`.

3 REFERENCES

2.3.3 Some Further Examples of JVM Instructions

In the following, *top* will refer to the element at the top of the Operand Stack, and *next-top* will refer to the element next to it.

- **aaload:**

The **aaload** instruction assumes that *next-top* is a pointer to an array (i.e. a variable declared of array type), and *top* is an index into the array. The instruction loads the array element and pushes it onto the Operand Stack. In other words, *next-top* and *top* are popped, and *next-top[*top*]* is fetched and pushed onto the Operand Stack.

The **aastore** instruction does the opposite.

- **aload**, etc.:

This pushes onto the Operand Stack a copy of the local variable in slot 0, which must be pointer to an array. The instructions **aload_1** and **aload_2** do the same for slots 1 and 2. For slot numbers greater than 2, the **aload** instruction is used, with *top* containing the slot number (which is popped).

The instructions **astore**, **astore_0** etc. do the opposite.

- **iconst_0**, etc.:

Pushes the integer constant 0 onto the Operand Stack. There are corresponding instructions up through **iconst_5**.

- **iload**, etc.:

The instructions **iload_0** through **iload_3** push onto the Operand Stack the values in local variable slots 0 through 3. For slot numbers greater than 3, there is **iload**, which has a second byte for the slot number, like **aload** above.

The instructions **istore**, etc. are similar.

- **isub**, etc.:

The subtraction instruction **isub** pops *next-top* and *top*, and pushes the difference *next-top* - *top*. Similarly for **iadd**, **imul** and **idiv**, with the latter pushing *next-top/top*.

- **if_icmpeq**:

Like **if_icmpge**, but tests for equality.

- **new**:

Performs the Java **new** operation. This is a three-byte instruction, with the second and third bytes being an index into the Constant Pool, pointing to the given class.

3 References

- Bill Venners' book, *Inside the Java Virtual Machine*. Available on the Web (with many helpful URL links) at www.artima.com.

3 REFERENCES

- Sun’s “official” definition of the JVM: *The Java Virtual Machine Specification*, by Lindholm and Yellin, also available on the Web, at <http://java.sun.com/docs/books/vmspec/html/VMSpecTOC.doc.html>. Chapter 6 gives specs (mnemonics, op codes, actions, etc.) on the entire JVM instruction set.